

DFS_BFS

```
graph={  
    'A':['B','C'],  
    'B':['D','E'],  
    'C':['F'],  
    'D':[],  
    'E':['F'],  
    'F':[]  
}
```

```
def dfs(visited,graph,node):  
    if node not in visited:  
        print(node)  
        visited.add(node)  
        for neighbour in graph [node]:  
            dfs(visited,graph,neighbour)
```

```
print("Depth first search of given graph: ")
```

```
visited=set()  
dfs(visited,graph,'A')
```

```
def bfs(visited2,graph,startNode):  
    visited2.append(startNode)  
    queue.append(startNode)  
    while queue:  
        s=queue.pop(0)  
        print(s,end=" ")  
        for neighbour in graph [s]:
```

```

        if neighbour not in visited2:
            visited2.append(neighbour)
            queue.append(neighbour)
print("\nBreadth first search of given graph: ")
visited2=[]
queue=[]
bfs(visited2,graph,'A')

```

A* for Puzzle problem code :

we use a tree data structure to solve thr a star problem

each node in the tree is an object of class Node

each object of class node is a form of the matrix

class Node:

```

    def __init__(self, data, level, fval):

```

```

        # Initialize the node with the data, level of the node and the calculated fvalue

```

```

        self.data = data

```

```

        self.level = level

```

```

        self.fval = fval

```

```

    def generate_child(self):

```

```

        """ Generate child nodes from the given node by moving the blank space

```

```

        either in the four directions {up,down,left,right} """

```

```

        x, y = self.find(self.data, '_')

```

```

        """ val_list contains position values for moving the blank space in either of

```

```

        the 4 directions [up,down,left,right] respectively. """

```

```

        val_list = [[x, y-1], [x, y+1], [x-1, y], [x+1, y]] #this are the x and y coordinates of the
        children in the matrix

```

```

        children = []

```

```

        #val list contain 4 values we check the values in the start game matrix

```

```

for i in val_list:

    child = self.shuffle(self.data, x, y, i[0], i[1])

    if child is not None: #we assign a child node if and only if the child exists in the matrix

        child_node = Node(child, self.level+1, 0)

        children.append(child_node)

return children

```

```

def shuffle(self, puz, x1, y1, x2, y2):

    # Move the blank space in the given direction and if the position value are out of limits
    the return None

    if x2 >= 0 and x2 < len(self.data) and y2 >= 0 and y2 < len(self.data):

        temp_puz = []

        temp_puz = self.copy(puz)

        temp = temp_puz[x2][y2]

        temp_puz[x2][y2] = temp_puz[x1][y1] #moving the blank space (exchange values in
matrix)

        temp_puz[x1][y1] = temp

        return temp_puz

    else:

        return None

```

```

def copy(self, root):

    # Copy function to create a similar matrix of the given node

    temp = []

    for i in root:

        t = []

        for j in i:

            t.append(j)

        temp.append(t)

    return temp

```

```

def find(self, puz, x):
    # Specifically used to find the position of the blank space
    for i in range(0, len(self.data)):
        for j in range(0, len(self.data)):
            if puz[i][j] == x:
                return i, j

```

```

class Puzzle:

```

```

    def __init__(self, size):
        # Initialize the puzzle size by the specified size, open and closed lists to empty
        self.n = size
        self.open = []
        self.closed = []

```

```

    def accept(self):
        # Accepts the puzzle from the user
        puz = []
        for i in range(0, self.n):
            temp = input().split(" ")
            puz.append(temp)
        return puz

```

```

    def f(self, start, goal):
        # Heuristic Function to calculate heuristic value  $f(x) = h(x) + g(x)$ 
        return self.h(start.data, goal) + start.level

```

```

    def h(self, start, goal):
        # Calculates the different between the given puzzles

```

```

temp = 0
for i in range(0, self.n):
    for j in range(0, self.n):
        if start[i][j] != goal[i][j] and start[i][j] != '_':
            temp += 1
return temp

```

```

def process(self):
    # Accept Start and Goal Puzzle state
    print("Enter the start state matrix \n")
    start = self.accept()
    print("Enter the goal state matrix \n")
    goal = self.accept()

    # we initiate the tree with the start configuration of the matrix
    start = Node(start, 0, 0)
    start.fval = self.f(start, goal)
    # Put the start node in the open list
    self.open.append(start)
    print("\n\n")
    while True:
        cur = self.open[0]
        print("")
        print(" | ")
        print(" | ")
        print(" \\\\/")
        print(" V \n")
        for i in cur.data:
            for j in i:

```

```

        print(j, end=" ")

    print("")

    # If the difference between current and goal node is 0 we have reached the goal node
    if(self.h(cur.data, goal) == 0):
        break

    for i in cur.generate_child():
        i.fval = self.f(i, goal)

        self.open.append(i)

    self.closed.append(cur)

    del self.open[0]

    # sort the opne list based on f value """
    self.open.sort(key=lambda x: x.fval, reverse=False)

puz = Puzzle(3)
puz.process()

```

Prim's Code :

```

def prim(graph):
    n = len(graph)
    selected = [False] * n
    selected[0] = True
    edges = []
    total_weight = 0

    for _ in range(n - 1):

```

```

min_edge = (None, None, float('inf'))
for u in range(n):
    if selected[u]:
        for v in range(n):
            if not selected[v] and graph[u][v] and graph[u][v] < min_edge[2]:
                min_edge = (u, v, graph[u][v])
u, v, w = min_edge
edges.append((u, v, w))
total_weight += w
selected[v] = True
return edges, total_weight

```

Example (adjacency matrix)

```

graph = [
    [0, 2, 0, 6, 0],
    [2, 0, 3, 8, 5],
    [0, 3, 0, 0, 7],
    [6, 8, 0, 0, 9],
    [0, 5, 7, 9, 0]
]

```

```

mst, total_weight = prim(graph)
for u, v, w in mst:
    print(f"{u} -- {v} : {w}")
print(f"Total weight of MST: {total_weight}")

```

Kruskal's code:

```

def kruskal(n, edges):
    parent = list(range(n))

    def find(u):
        while parent[u] != u:
            parent[u] = parent[parent[u]]
            u = parent[u]
        return u

    mst = []
    total_weight = 0
    edges.sort() # Sort by weight

    for w, u, v in edges:
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            mst.append((u, v, w))
            total_weight += w

    return mst, total_weight

# ----- USER INPUT SECTION -----
n = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

edges = []
print("Enter each edge as: u v weight (space separated, 0-based indexing)")
for _ in range(m):

```

```

u, v, w = map(int, input().split())
edges.append((w, u, v)) # Note: weight comes first for sorting

# ----- RUN KRUSKAL -----
mst, total = kruskal(n, edges)

# ----- OUTPUT -----
print("\nEdges in constructed MST")
print("MST:", mst)
print("Minimum Spanning Tree Total Weight:", total)

```

N_queen problem :

```

N = int(input("Enter the number queen: "))
board = [[0]*N for _ in range(N)]

def is_attack(i, j):
    for k in range(0, N):
        if board[i][k] == 1 or board[k][j] == 1:
            return True
    for k in range(0, N):
        for l in range(0, N):
            if (k+l == i+j) or (k-l == i-j):
                if board[k][l] == 1:
                    return True
    return False

```

```

def N_queen5(n):
    if n == 0:
        return True
    for i in range(0, N):
        for j in range(0, N):
            if (not (is_attack(i, j))) and (board[i][j] != 1):
                board[i][j] = 1
                if N_queen5(n-1) == True:
                    return True
                board[i][j] = 0
    return False

```

```

N_queen5(N)
for i in board:
    print(i)

```

chatbot code :

```

import random

# define the possible user inputs and corresponding chatbot responses
responses = {
    "hi": ["Hello!", "Hi there!", "Hi, how can I help you?"],
    "hello": ["Hello there!", "Hi! How can I assist you today?"],
    "how are you": ["I'm doing well, thank you. How about you?",
                    "I'm great, thanks for asking! How can I assist you?"],
    "what's your name": ["My name is Chatbot. How may I assist you?",

```

```
        "I'm Chatbot. How can I help you today?"],
    "goodbye": ["Goodbye!", "Have a great day!"],
    "default": ["Sorry, I didn't understand your question. Can you please rephrase?",
        "I'm not sure what you mean. Can you please clarify?"],
}
```

```
# define a function to generate a chatbot response based on user input
```

```
def generate_response(user_input):
```

```
    # convert user input to lowercase for easy matching
```

```
    user_input = user_input.lower()
```

```
    # find a response for the user input, or use a default response if not found
```

```
    chatbot_response = responses.get(user_input, responses["default"])
```

```
    # randomly select a response from the possible options
```

```
    return random.choice(chatbot_response)
```

```
# main program loop
```

```
while True:
```

```
    # get user input
```

```
    user_input = input("You: ")
```

```
    # generate chatbot response based on user input
```

```
    chatbot_response = generate_response(user_input)
```

```
    # print chatbot response
```

```
    print("Chatbot:", chatbot_response)
```

```
    # exit loop if user says goodbye
```

```
    if user_input.lower() == "goodbye":
```

```
        break
```

Graph coloring Code:

```
G = [[0, 1, 1, 0, 1, 0],  
      [1, 0, 1, 1, 0, 1],  
      [1, 1, 0, 1, 1, 0],  
      [0, 1, 1, 0, 0, 1],  
      [1, 0, 1, 0, 0, 1],  
      [0, 1, 0, 1, 1, 0]]
```

```
# Initiate the names of the nodes
```

```
node = "abcdef"
```

```
t_ = {}
```

```
for i in range(len(G)):
```

```
    t_[node[i]] = i
```

```
# Count degree of all nodes
```

```
degree = []
```

```
for i in range(len(G)):
```

```
    degree.append(sum(G[i]))
```

```
# Initiate the possible colors
```

```
colorDict = {}
```

```
for i in range(len(G)):
```

```
    colorDict[node[i]] = ["Blue", "Red", "Yellow", "Green"]
```

```
# Sort the nodes based on degree using selection sort
```

```
sortedNode = []
```

```
indeks = []
```

```

for i in range(len(degree)):
    _max = -1
    for j in range(len(degree)):
        if j not in indeks and degree[j] > _max:
            _max = degree[j]
            idx = j
    indeks.append(idx)
    sortedNode.append(node[idx])

# Main coloring process
theSolution = {}

for n in sortedNode:
    setTheColor = colorDict[n]
    theSolution[n] = setTheColor[0]
    adjacentNode = G[t_[n]]
    for j in range(len(adjacentNode)):
        if adjacentNode[j] == 1 and (setTheColor[0] in colorDict[node[j]]):
            colorDict[node[j]].remove(setTheColor[0])

# Print the solution
for t, w in sorted(theSolution.items()):
    print("Node", t, "=", w)

```